

Memory in Memory: Learning Procedural Memory Policies from Downstream Failures

Anonymous Author(s)

Contact information omitted for anonymization

August 2026

Abstract

Large language model (LLM) agents increasingly rely on external memory to preserve user facts, state, and history across sessions. Existing memory systems primarily improve the *representation*, *compression*, and *retrieval* of content, but rarely study how a memory system can continuously learn “how it should remember” from its own downstream errors. When a question is answered incorrectly, the same failure signal can originate from completely different stages of the pipeline: the target information may never have been written into memory, it may exist but fail to be retrieved, or it may already be in context yet be misused by the answer model. Methods that reflect over failures or rewrite global prompts without separating these root causes risk misattribution, duplicated rules, and global regressions.

We propose **Memory in Memory** (MiM), an error-driven *procedural meta-memory* framework for LLM-mediated memory systems. MiM decomposes a long-term memory system into a memory construction policy and a memory access policy, and adds an offline error-learning loop on top of normal operation. Given a downstream failure, MiM runs three information-isolated checks—answer evidence sufficiency, access coverage of currently useful memory, and construction fidelity along provenance and version chains—so that access omissions and construction losses are attributed as non-exclusive, causally scoped failures rather than a single miscellaneous label. Confirmed failures are distilled into natural-language *candidate skills*, which are semantically clustered, related to the existing skill bank through an explicit candidate–bank affinity matrix, and integrated by a constrained batch CRUD process with write-set conflict resolution before being released as a new versioned skill bank. At runtime, only frozen official skills are retrieved, while a skill-exposure trace records both the skills actually injected and the boundary candidates near the retrieval margin, enabling later learning rounds to distinguish policy absence, recall failure, and execution ineffectiveness. MiM thereby lets a long-term memory system accumulate not only object-level facts about users and the world, but also procedural experience about how its own construction and access policies should behave.

1 Introduction

External memory has become a foundational component of long-running LLM agents [?, ?]. An agent equipped with persistent memory can retain user preferences, personal relationships, historical events, and state changes across sessions, and selectively recover the relevant information in future tasks [?, ?, ?]. Toward this goal, a rich design space has emerged: vector memory, hierarchical summaries, structured facts, temporal knowledge graphs, multi-type memory, and agent-managed memory systems [?, ?, ?, ?, ?, ?]. Despite ever-stronger underlying representations, long-term memory systems still exhibit recurring failure patterns: important information is not retained, updates destroy historical state, semantically similar but distinct events are incorrectly merged, a

fact that exists in memory is never retrieved, or a multi-hop question is answered prematurely with only partial evidence.

These errors do not originate solely from storage capacity or retriever performance. For an *LLM-mediated* memory system, the decisive decisions occur at the policy level: the model decides which information is worth writing, how new information should be integrated with existing memory, what should be retrieved for the current question, whether to keep searching, and when the accumulated evidence is sufficient to answer. Even if the underlying store can persist and return data faithfully, inappropriate construction and access policies produce systematic failures.

Existing systems typically accumulate *object-level* memory, that is, facts about users, environments, and history. In contrast, they rarely accumulate *procedural meta-memory*, i.e., experience about the memory system’s own operation. For example, a system may record “the user has moved to Shanghai” but never learn from a previous mistake that “when updating the current address, the historical validity interval of the old address should be preserved”; it may store several related events but never learn that “for list questions, one should not stop retrieving after finding only a single component.”

A straightforward remedy is to save failure cases and retrieve past errors when a similar problem arises, or to append every reflection to the global prompt. Both approaches lack a critical *root-cause constraint*. An incorrect answer is only a downstream symptom: the target information may have failed to enter memory, failed to be retrieved, or been visible yet misused. If the system generalizes an access omission into a construction rule, or writes an answer reasoning mistake as a retrieval policy, the new experience will not only fail to transfer but may also harm unrelated tasks.

Learning memory policies from errors therefore first requires answering a causal question:

In the pipeline spanned by raw interaction, memory state, access trajectory, and answer context, where is the first point at which the target information is no longer correctly preserved or used?

We propose **Memory in Memory** (MiM), which turns downstream errors into attributable, maintainable, and reusable procedural memory policies. MiM does not replace the underlying memory system; it adds an external *control plane*. During normal operation, the base system constructs and accesses memory exactly as before. In an offline learning phase, MiM observes the raw interaction, memory versions, natural search trajectories, and answer outputs, and diagnoses three families of failure: evidence misuse at answer time, access omission over the current memory state, and construction loss along the version history.

MiM’s diagnosis is not a single mutually exclusive label produced by one model call. Answer sufficiency, access omission, and construction loss are modeled as three checks over *disjoint information views*. The access check compares the currently useful memory and the natural search results on the fixed current snapshot, without reading the raw conversation or historical versions; the construction check first verifies whether the current memory is complete and only then walks the source and version relations to locate the earliest corruption. A single question can carry both access and construction problems, because the current memory may have lost one fact while simultaneously failing to recall another fact that is still present.

For confirmed access or construction failures, MiM does not inject the full failure case into the running context. Instead it distills a short, retrievable, natural-language *candidate skill* that describes when a class of situations occurs and what operational principle the memory agent should follow. To prevent per-case updates from inflating the policy library, MiM first induces candidates independently, then clusters semantically similar candidates, retrieves the related official skills, and executes a constrained batch CRUD consolidation. Only after global integration and consistency validation does a candidate become an official, runtime-retrievable policy.

MiM also records, for every decision, which official skills were actually retrieved and which candidates sat near the retrieval boundary. This *skill-exposure trace* lets the next learning round distinguish three different failure types: no relevant experience exists in the policy bank, a relevant experience exists but did not enter the context, and a relevant experience was used but its content was ineffective. Skill learning thus ceases to be an unobservable prompt rewrite and becomes a closed loop whose representation, recall, and execution stages can be analyzed separately.

Our core contributions are as follows:

1. We introduce the problem setting of *error-driven procedural meta-memory*: long-term memory systems should accumulate reusable experience about their own construction and access policies, not only object-level facts.
2. We propose an *information-isolated failure attribution* method that decomposes answer insufficiency, access omission, and construction loss into non-exclusive checks, and uses source and version relations to locate the first state transition at which information fidelity is broken.
3. We propose *direction-separated dual skill banks* for access and construction, together with a two-stage promotion mechanism from instance-level candidates to official frozen policies.
4. We propose a *constrained skill-bank evolution* procedure based on candidate–bank relations, batch CRUD with a deterministic validator, and write-set conflict resolution, in order to control duplication, order dependence, and version oscillation.
5. We build a fully observable long-term-memory research base and define a conversation-level frozen evaluation protocol that measures final task quality, error-type changes, and natural skill invocation separately.

The remainder of this paper is organized as follows. Section ?? discusses related work on agent memory, self-evolving memory policies, and error-driven agent improvement. Section ?? formalizes LLM-mediated memory systems, object-level versus procedural memory, and the failure signal. Section ?? presents the MiM framework in detail, including traceable memory states, closed-loop access, isolated failure attribution, candidate induction, semantic consolidation, and constrained bank evolution. Section ?? concludes.

2 Related Work

2.1 Memory Architectures for LLM Agents

Persistent memory for LLM agents has been studied along several axes. **Tiered memory** organizes information by access frequency and lifetime: MemGPT (Letta) [?] borrows the operating-system metaphor of working memory and long-term storage and exposes memory manipulation as function calls, making access decisions part of the agent’s tool-use loop. **Structured and associative memory** represents facts as knowledge graphs or associative networks, as in Mem0 [?], A-MEM [?], and Zep/Graphiti [?], which build temporal knowledge graphs that support time-aware updates. **Multi-type memory** distinguishes episodic, semantic, procedural, and other categories: MemoryOS [?] layers short-term, mid-term, and long-term stores; MIRIX [?] routes inputs through six specialized memory managers supervised by a meta manager and pairs them with an access agent that selects memory components and retrieval methods per question. **Efficient memory** targets token economy: LightMem [?] and SimpleMem [?] compress interaction histories into retrieval-friendly units, and Hindsight [?] maintains four typed networks with retain/recall/reflect operations. Recent

surveys [?] formalize agent memory as a write–manage–read loop, a perspective that positions policy control—what to write, how to manage, and how to read—as the core design decision.

These systems improve *what* is stored and *how* it is retrieved, but the policies that govern construction and access are largely fixed at deployment. MiM complements this line by treating construction and access *policies* themselves as a learnable resource: rather than proposing a new storage representation, it attaches an error-learning loop that accumulates procedural experience about any underlying memory architecture.

2.2 Skill-Guided and Self-Evolving Memory Policies

A recent line of work makes memory policies learnable. **MemSkill** [?] reframes memory construction operations as a small set of reusable, evolvable skills: a controller selects relevant skills, an executor produces skill-guided memories, and a designer periodically reviews hard cases to refine or create skills. MemSkill is the closest neighbor on the construction side, but its skill selection is trained with reinforcement learning and it does not learn a multi-tool, stop-capable access policy. **EvolveMem** [?] exposes the full retrieval configuration (scoring functions, fusion weights, budgets) as an action space and evolves it with an LLM-powered diagnosis module under revert-on-regression guards. It targets continuous configuration optimization rather than discrete, retrievable, natural-language policies. **MemMA** [?] places a meta-thinker and query reasoner outside a unified storage interface: it diagnoses evidence gaps per question, generates orthogonal queries, and directly repairs factual entries on the current instance. MemMA does not accumulate cross-instance procedural skills; its controller improves the current question rather than the agent’s future construction and access behavior. **PlugMem** [?] is a task-agnostic plugin controller for memory access, selecting memory types and planning retrieval, but it is designed as a replaceable module rather than a bank of experience learned from failures. On the access side, **MRAgent** [?] and **MemMachine** [?] build strong multi-tool retrieval agents over graph or episode stores, yet their strategies are frozen after design.

MiM differs from this line in two ways. First, it is *error-driven and attribution-first*: nothing is learned unless a downstream failure has been isolated to the access or construction stage, so repair targets are causally scoped. Second, its skills are *procedural meta-memory*: versioned, retrievable natural-language policies that are integrated through constrained batch CRUD, rather than continuous configuration deltas or per-instance fact repairs.

2.3 Error-Driven Agent Improvement and Reflection

Reflection-based methods let an agent improve from its own mistakes. Reflexion [?] converts failed trajectories into verbal feedback stored in an episodic buffer; Self-Refine [?] iteratively revises outputs; Generative Agents [?] maintain episodic buffers indexed by recency and importance; MemoryBank [?] applies forgetting curves and periodic consolidation to prune stale entries. Prompt-level optimization, such as OPRO [?] and the prompt optimizer shipped with LangMem [?], rewrites the agent’s policy prompt globally from trajectory feedback.

These methods share a common weakness that MiM targets directly: reflection is applied *without separating where the pipeline first failed*. A global prompt rewrite triggered by an access failure may modify construction behavior; a reflection over an answer reasoning error may produce a retrieval rule. MiM instead attributes each failure to answer, access, or construction stages first, and only then induces a directional skill that is retrieved conditionally at runtime, so new experience is applied where its root cause lives and remains reusable without contaminating unrelated tasks.

2.4 Skill Libraries for Agents

Beyond memory, agents increasingly reuse *procedural skills*. Voyager [?] maintains a growing library of executable code skills acquired through environment feedback, and recent agent frameworks treat skills as versioned, retrievable units of behavior. MiM shares the spirit of *learning reusable procedures rather than facts*, but with a different substrate: its skills are concise natural-language operation principles for memory construction and access, induced from offline failure attribution, and maintained through a deterministically validated, conflict-free batch evolution process. This makes skill quality inspectable and the bank versionable, both of which are prerequisites for a memory system whose learning must not compromise the reliability of the object-level store it guides.

3 Problem Formulation

3.1 LLM-Mediated Memory Systems

We consider a long-term memory system whose construction and access decisions are at least partially made by LLMs:

$$\mathcal{M} = (\mathcal{D}, \pi_{\text{con}}, \pi_{\text{acc}}, f_{\text{ans}}), \quad (1)$$

where $\mathcal{D}$ is the persistent memory state, π_{con} is the memory *construction policy*, π_{acc} is the memory *access policy*, and f_{ans} is the task model that produces an answer from the question and the visible evidence.

Given the t -th interaction segment x_t and the previous memory state D_{t-1} , the construction policy produces a new state and a construction trajectory:

$$D_t, \tau_t^{\text{con}} = \pi_{\text{con}}(x_t, D_{t-1}; S_t^{\text{con}}), \quad (2)$$

where S_t^{con} is the set of construction skills available at this round and τ_t^{con} is the construction trajectory (the sequence of candidate proposals, decisions, and state transitions).

Given a question q , the access policy interacts with memory through a sequence of actions $a_1, \dots, a_J$:

$$a_j \sim \pi_{\text{acc}}(a_j \mid q, D_t, S_q^{\text{acc}}, h_{<j}), \quad (3)$$

where S_q^{acc} is the set of access skills retrieved for q , and $h_{<j}$ contains the previous queries, tool observations, and intermediate judgments. When the policy executes a stop action, the answer model produces a prediction from the accumulated visible evidence M_{visible} :

$$\hat{y} = f_{\text{ans}}(q, M_{\text{visible}}). \quad (4)$$

In our current implementation, access and answering are performed by the same closed-loop agent, so the stop decision and the final answer share the full natural search context. We revisit this choice in Section ??.

3.2 Object-Level and Procedural Memory

Object-level memory D_t stores facts about users and the world. MiM maintains an additional, separate resource: two *procedural policy banks*

$$\mathcal{K} = (\mathcal{K}_{\text{con}}, \mathcal{K}_{\text{acc}}), \quad (5)$$

where $\mathcal{K}_{\text{con}}$ guides information selection, extraction, updating, merging, and historical preservation during construction, and $\mathcal{K}_{\text{acc}}$ guides query planning, retrieval routes, filtering, evidence aggregation, and stopping during access.

Each skill uses a minimal natural-language representation

$$s = (n, d, c), \quad (6)$$

where n is the skill name, d is the retrieval description (“when does this skill apply”), and c is the execution content (“what principle should be followed”). Skills do not retain user facts from training cases; they describe processing principles that transfer across entities and sessions.

3.3 Failure Signal

Given a reference answer y^* and the model prediction $\hat{y}$, a semantic judge yields

$$J(\hat{y}, y^*) \in \{\text{C}, \text{P}, \text{I}\}, \quad (7)$$

where C, P, and I denote correct, partial, and incorrect, respectively. Only P and I enter failure attribution. The judge observes only the question, the reference answer, the prediction, and the conversation time anchor; it does not read the memory state or the search trajectory, so it confirms a downstream failure without explaining it.

Let $\text{Cover}(M, y^*)$ denote the degree to which an evidence set M covers the necessary facts of y^* , and $\text{Support}(m, y^*)$ indicate whether a memory entry m supports at least one necessary fact. MiM defines three diagnosis outcomes.

Definition 3.1 (Answer failure). *An answer failure occurs when the prediction is not correct even though the evidence visible to the answer model was already sufficient:*

$$F_{\text{ans}} = \mathbb{I}[J(\hat{y}, y^*) \neq \text{C} \wedge \text{Cover}(M_{\text{visible}}, y^*) = 1]. \quad (8)$$

Definition 3.2 (Access failure). *Let the currently useful memory be*

$$M_{\text{useful}} = \{m \in D_t : \text{Support}(m, y^*) = 1\}, \quad (9)$$

and let the memories returned by the natural search chain be

$$M_{\text{retrieved}} = \bigcup_{j=1}^J M_j. \quad (10)$$

An access failure occurs when useful memory exists but was not retrieved:

$$F_{\text{acc}} = \mathbb{I}[M_{\text{useful}} \setminus M_{\text{retrieved}} \neq \emptyset]. \quad (11)$$

Definition 3.3 (Construction failure). *A construction failure occurs when the raw interaction supports the necessary facts but the current memory state no longer covers them:*

$$F_{\text{con}} = \mathbb{I}[\text{Support}(X_{\text{raw}}, y^*) = 1 \wedge \text{Cover}(D_t, y^*) < 1], \quad (12)$$

where X_{raw} denotes the raw source evidence.

These outcomes are not mutually exclusive. In particular, F_{acc} and F_{con} can hold simultaneously: the current memory may have lost one fact (construction) while failing to recall another fact that is still present (access). MiM therefore never reduces a failure to a single label; the three checks are computed independently over disjoint information views (Section ??).

Algorithm 1 Failure-driven skill learning (training phase)

Require: conversations, questions, train split, frozen bank $\mathcal{K}^{(v)}$

```
1: Phase 1 – run.
2: for conversation  $c$  in train split do
3:    $D_t \leftarrow \text{INGEST}(c; \mathcal{K}_{\text{con}}^{(v)})$  ▷ batch CRUD, single transaction
4:   for question  $q$  in  $c$  do
5:      $(\hat{y}, M_{\text{visible}}, \tau^{\text{acc}}) \leftarrow \text{ASK}(q; \mathcal{K}_{\text{acc}}^{(v)})$ 
6:     record skill-exposure trace  $T_q = (\mathcal{S}_q^{\text{selected}}, \mathcal{S}_q^{\text{nearby}}, v)$ 
7:   end for
8: end for
9: Phase 2 – attribute.
10: for failed question with  $J(\hat{y}, y^*) \neq \text{C}$  do
11:    $F_{\text{ans}}$ : answer check on  $M_{\text{visible}}$  only
12:    $F_{\text{acc}}$ : useful vs. retrieved on current snapshot  $D_t$ 
13:    $F_{\text{con}}$ : screen  $D_t$ ; if gap found, trace first error  $t^*$ 
14:   for side  $z \in \{\text{acc}, \text{con}\}$  with confirmed failure do
15:      $c_i \leftarrow G_{\text{cand}}(d_i, T_i)$  ▷ or  $c_i = \emptyset$  (no change)
16:   end for
17: end for
18: Phase 3 – consolidate and publish.
19:  $(\mathcal{K}_{\text{acc}}^{(v+1)}, \mathcal{K}_{\text{con}}^{(v+1)}) \leftarrow \text{CONSOLIDATE}(\text{candidates}, \mathcal{K}^{(v)})$  ▷ Alg. ??
20: select bank version on validation split; freeze for evaluation
```

4 Method: Memory in Memory

4.1 Overview

MiM adds an error-learning control plane outside the base memory system. One complete iteration consists of three phases:

1. the base system constructs memory and answers training questions under a *frozen* skill bank;
2. MiM runs information-isolated root-cause diagnosis over semantic failures and induces candidate skills;
3. candidates are globally consolidated into a new skill-bank version, which is selected on validation data and then frozen for subsequent runs.

This design separates three concerns that are usually conflated. *Diagnosis* answers “what does this failure require learning”; *consolidation* answers “how does this repair relate to existing policies”; *frozen evaluation* answers “does the updated bank work on unseen sessions.” Algorithm ?? summarizes the training-time loop.

4.2 Traceable Memory States

Attributing construction failures requires that the memory system expose source and version relations. We represent each memory state as

$$D_t = (V_t, E_{\text{src}}, E_{\text{par}}, E_{\text{chg}}), \quad (13)$$

where V_t is the set of memory versions visible at system time t , E_{src} connects versions to their originating raw messages, E_{par} denotes parent-child relations introduced by updates or merges, and E_{chg} records the state transitions that produced new versions. Each transition is generated by an operation

$$o_t \in \{\text{ADD, UPDATE, MERGE, DELETE, SKIP}\}, \quad (14)$$

and source relations are inherited across updates, so the system can trace any raw message to the candidate it first formed, its initial version, and every subsequent change.

This representation lifts construction diagnosis from “is the final memory wrong” to “at which state transition was the information first corrupted”. Let $\mathcal{I}(D_t, X)$ be the invariant that memory state D_t still faithfully preserves the target information from source evidence X . The earliest error point is

$$t^* = \min \{ t : \mathcal{I}(D_{t-1}, X) = 1 \wedge \mathcal{I}(D_t, X) = 0 \}. \quad (15)$$

If the initial extraction was already wrong, t^* corresponds to candidate formation; if the initial state was correct, t^* corresponds to the first update, merge, or deletion that broke the invariant.

4.3 Closed-Loop Memory Access

MiM does not model access as a single fixed retrieval. The access policy chooses, after every observation, between searching, inspecting, and answering. At round j , the policy receives the previous history $h_{<j}$, emits action a_j , receives tool observation o_j , and updates

$$h_j = h_{j-1} \cup \{a_j, o_j\}. \quad (16)$$

Search actions can choose among semantic, lexical, exact-match, and structured temporal routes. For hybrid retrieval, the candidate score is

$$s(m \mid q) = \sum_{r \in \mathcal{R}} \frac{w_r}{k + \text{rank}_r(m)}, \quad (17)$$

where $\mathcal{R}$ is the set of retrieval routes with weights w_r summing to one (reciprocal-rank fusion), and is then corrected by entity matching, target time validity, and current-state activity factors.

The agent judges, from the accumulated evidence, whether the necessary facts are complete. If information is insufficient, it can decompose the question, change the query formulation, switch routes, or deepen the search; if sufficient, it stops immediately. This closed loop gives access skills a real control target: a skill can influence not only the final query text but also the retrieval route, the continuation condition, and the evidence-completeness judgment.

4.4 Isolated Failure Attribution

MiM builds three independent information views for the same failure.

Answer view. The answer check sees only the evidence actually visible to the running model, and determines whether the error occurred at the evidence-utilization stage (Definition ??). It produces no repair package and does not enter skill learning; it is recorded so that failures are never misattributed to memory when the memory evidence was already sufficient.

Access view. The access check is fixed to the current memory snapshot and contains only the currently relevant memory and the natural access trajectory. It does not read the raw conversation or historical versions, so that upstream construction errors are not introduced into access attribution. A semantic model labels the useful entries, and a deterministic program computes the difference

$$M_{\text{missing}} = M_{\text{useful}} \setminus M_{\text{retrieved}}. \quad (18)$$

Construction view. The construction check uses progressive disclosure. The first stage examines only whether the current relevant memory covers the necessary facts (FULL/PARTIAL/MISSING/INCORRECT); only when a problem is found does the second stage open the raw source messages and the version trajectory to locate t^* via Eq. (??).

This permission design is not about shrinking model inputs; it establishes an explanatory causal boundary. An access failure means “available information was not retrieved,” while a construction failure means “the current information state itself is wrong.” Because the three workflows are physically and contextually isolated, they can run in parallel and produce independent repair targets.

4.5 Runtime Skill Exposure

At runtime, the agent retrieves skills from the direction-corresponding bank:

$$\mathcal{S}_q = \text{TopK}_{s \in \mathcal{K}_z} [\lambda \cdot \text{sim}_{\text{sem}}(q, s) + (1 - \lambda) \cdot \text{sim}_{\text{lex}}(q, s)], \quad z \in \{\text{acc}, \text{con}\}, \quad (19)$$

where semantic similarity is computed over the skill name and description (the trigger condition), and lexical similarity is a BM25 score. A fixed candidate pool is drawn first, then an LLM-based applicability reranker filters out skills that do not actually apply to the current decision, and a score threshold rejects low-confidence matches. In our configuration, $\lambda = 0.7$, at most two skills are injected per decision, and the five next-ranked boundary candidates are retained.

Besides the injected skills, the system records the candidates near the retrieval boundary that did not enter the context:

$$T_q = (\mathcal{S}_q^{\text{selected}}, \mathcal{S}_q^{\text{nearby}}, v_{\mathcal{K}}), \quad (20)$$

where $v_{\mathcal{K}}$ is the official bank version. T_q is the policy exposure trace consumed by later learning rounds. If a relevant skill already entered $\mathcal{S}_q^{\text{selected}}$ and the failure persists, the problem most likely lies in the skill content or its execution; if it appears only in $\mathcal{S}_q^{\text{nearby}}$, the problem is likely in the retrieval representation; if it appears in neither, the system may need to learn a new policy. These three cases—policy absence, recall failure, and execution ineffectiveness—are thereby separable, which is precisely what a prompt-level reflection method cannot observe.

4.6 Failure-Conditioned Skill Induction

For each diagnosis package d_i and its policy exposure trace T_i , a candidate agent produces

$$c_i = \text{G}_{\text{cand}}(d_i, T_i), \quad (21)$$

where c_i contains a name, a retrieval description, execution content, and a short statement of the general problem it solves. The generation process is required to remove concrete entities, dates, answers, and internal IDs so that the candidate remains reusable across cases.

The candidate agent may also output

$$c_i = \emptyset, \quad (22)$$

indicating that existing policies already cover the problem, or that the problem is not suitable for repair through a memory skill. The empty operation is a deliberate mechanism for controlling bank growth, not a generation failure.

Candidates cannot be retrieved by the runtime before publication. MiM therefore establishes a two-stage policy lifecycle: instance-level failures first produce non-executable repair hypotheses, and a global consolidation process decides their creation, merging, update, or rejection.

4.7 Semantic Candidate Consolidation

Updating the policy bank error-by-error introduces strong order dependence: a skill released for one error changes the bank seen by the next error, similar failures may create duplicate rules, and the same skill may be rewritten repeatedly. MiM therefore collects candidates first and integrates them in batches.

The semantic representation of a candidate is

$$e(c_i) = \alpha e_{\text{desc}}(c_i) + \beta e_{\text{content}}(c_i) + \gamma e_{\text{solves}}(c_i), \quad (23)$$

with $\alpha + \beta + \gamma = 1$ (in our configuration, 0.45/0.35/0.20). Semantically similar candidates are grouped by deterministic spherical K -means into candidate clusters $\{\mathcal{C}_g\}_{g=1}^G$, subject to a target cluster size and a maximum batch size.

For a candidate c_i and an existing skill s_j , MiM computes the relation explicitly:

$$R_{ij} = \lambda_d \text{sim}_{\text{sem}}(c_i^d, s_j^d) + \lambda_c \text{sim}_{\text{sem}}(c_i^c, s_j^c) + \lambda_l \text{sim}_{\text{lex}}(c_i, s_j), \quad (24)$$

with $\lambda_d + \lambda_c + \lambda_l = 1$ (in our configuration, 0.50/0.30/0.20). Compared with concatenating an entire group of candidates into a single query, the matrix R preserves each repair hypothesis’s own related skills while still allowing the discovery of a common policy that covers multiple candidates. Before planning, candidates whose best relation score exceeds a threshold (0.60 in our configuration) are linked to the corresponding official skills.

4.8 Constrained Skill-Bank Evolution

For a candidate cluster $\mathcal{C}_g$, the related existing skills, and the relation matrix R_g , a CRUD agent produces an update plan

$$P_g = G_{\text{crud}}(\mathcal{C}_g, \mathcal{K}_z^{(v)}, R_g). \quad (25)$$

The plan can add a skill, rename a skill, update a description, add or update content, move content, or deactivate a skill. Every candidate must receive a unique resolution, e.g., created, merged into an existing skill, covered by an existing skill, or rejected. The model only proposes plans; a deterministic validator checks target existence, direction, expected bank version, content positions, and candidate coverage:

$$\text{Valid}(P_g, \mathcal{K}_z^{(v)}) = 1. \quad (26)$$

Only valid plans may modify the bank.

Different candidate clusters can be planned in parallel against the same frozen version. Let W_g be the write set of plan P_g (the set of skills it modifies). If

Algorithm 2 Consolidation: candidate clusters $\rightarrow$ new bank version

Require: candidates $\{c_i\}$, official bank $\mathcal{K}_z^{(v)}$ (frozen)

- 1: cluster candidates: $\{\mathcal{C}_g\}_{g=1}^G \leftarrow \text{Cluster}(\{e(c_i)\})$
- 2: **for** each cluster $\mathcal{C}_g$ **do**
- 3: retrieve candidate–bank relations R_g ; link best relations ≥ 0.60
- 4: $P_g \leftarrow \text{G}_{\text{crud}}(\mathcal{C}_g, \mathcal{K}_z^{(v)}, R_g)$ $\triangleright$ retry up to 3 times
- 5: **if** $\text{Valid}(P_g, \mathcal{K}_z^{(v)}) = 0$ **then**
- 6: reject cluster
- 7: **end if**
- 8: **end for**
- 9: **repeat**
- 10: build conflict graph: edge (i, j) iff $W_i \cap W_j \neq \emptyset$
- 11: **for** each connected component with ≥ 2 plans **do**
- 12: merge candidates, re-retrieve, re-plan jointly
- 13: **end for**
- 14: **until** no conflicts
- 15: run publication quality gate (payload validation, candidate support)
- 16: $\mathcal{K}_z^{(v+1)} \leftarrow \text{Apply}(\mathcal{K}_z^{(v)}, \bigcup_g P_g)$ **return** $\mathcal{K}_z^{(v+1)}$

$$W_i \cap W_j \neq \emptyset, \quad (27)$$

the two plans conflict. MiM builds the conflict graph and re-generates a joint plan for each connected component, re-retrieving the combined candidate set against the official bank. After all conflicts are resolved, the updates are applied in a single transaction to produce a new bank version:

$$\mathcal{K}_z^{(v+1)} = \text{Apply}\left(\mathcal{K}_z^{(v)}, \bigcup_g P_g\right). \quad (28)$$

This procedure combines the LLM’s semantic integration ability with deterministic state management: the model decides how policy content is organized, while the program guarantees consistency and traceability of the update. Algorithm ?? summarizes the pipeline.

4.9 Validation and Frozen Evaluation

MiM induces candidates on training failures, selects a bank version on validation sessions, and freezes it for testing:

$$v^* = \arg \max_v \text{F1}_{\text{val}}(\mathcal{K}^{(v)}), \quad (29)$$

where F1_{val} is the token-F1 on the validation split, computed under the same frozen protocol as the test. At test time, no reference answers are provided, no diagnosis runs, and the bank is never updated. The base system and MiM use identical models, storage, tools, steps, and token budgets; the only difference is the retrieval and injection of frozen skills. To detect potential answer leakage, skills are audited for concrete entities, dates, internal IDs, and rare strings, and evaluation is performed on a conversation-disjoint split (6:2:2 in our setup), so that no persona, history, or event crosses split boundaries.

Beyond maximizing task quality, MiM’s long-term objective also controls the number of policies, redundancy, and update oscillation:

$$\max_{\mathcal{K}} \quad Q(\mathcal{K}) - \alpha |\mathcal{K}| - \beta \text{Redundancy}(\mathcal{K}) - \eta \text{Churn}(\mathcal{K}) - \mu \text{Cost}(\mathcal{K}), \quad (30)$$

where $Q(\mathcal{K})$ is task quality under the frozen bank. The current method approximates this objective through discrete candidates, batch CRUD, and version selection; it does not rely on gradient training.

4.10 Design Rationale

Why separate construction and access skills. Construction and access act on different states. A construction skill changes future memory states, so a bad update can affect many later questions; an access skill modifies no memory and affects only how the current question obtains evidence. Their observable traces, failure conditions, and validation methods differ as well. A single shared bank lets rules with similar vocabulary but different causal positions interfere—e.g., “handle multiple temporal versions” may mean either preserving a validity interval during construction or selecting the target time during access. Direction-separated banks keep repair responsibility clear.

Why merge access and answer into one loop. Whether access should continue depends on the model’s understanding of current evidence. If an access agent produced a fixed evidence set and handed it to a separate answer agent, the latter could not dynamically complete missing information. MiM places retrieval and answering in the same ReAct context so that the model can judge FULL/PARTIAL/NONE after each observation and stop autonomously. This merge does not affect attribution: the offline answer check still examines whether the information ultimately visible to the running model was already sufficient.

Why use gold annotations only offline. Reference answers and evidence serve as offline supervision to judge failures and to locate repair targets during training. The final test provides no gold and allows no error-driven updates. This mirrors supervised training of model parameters: gold is used to learn policies, but the frozen system must retrieve and execute them naturally on unseen sessions.

Why batch candidate consolidation. A candidate is a local repair hypothesis induced from a single failure and does not necessarily correspond to an independent long-term skill: multiple errors may share one cause, and one error may already be covered by an existing skill. Batching first preserves the independence of instance-level induction and then applies global compression, which reduces the influence of sample order on bank structure and provides explicit measurement objects for duplication rate, merge rate, and version oscillation.

5 Conclusion

This paper studies how long-term memory systems can learn reusable memory policies from their own errors. MiM decomposes downstream failures into answer evidence utilization, current-memory access, and historical-memory construction problems, and uses source, version, and natural search trajectories to form auditable repair targets. Confirmed access and construction failures are distilled into direction-separated candidate skills (access and construction), which become an official skill bank through global relation modeling, conflict-constrained evolution, and versioned release. The framework thereby extends the object of long-term accumulation from “facts about users and the

world” to “procedural experience about how the system itself should construct and access memory,” and turns policy presence, recall, and effectiveness into separately observable and evaluable processes.

References

- [1] P. Du, “Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers,” arXiv:2603.07670, 2026.
- [2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [3] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023.
- [4] W. Zhang, J. Deng, X. Zheng, and X. Zhu, “A-MEM: Agentic memory for LLM agents,” arXiv:2502.12110, 2025.
- [5] D. Chattopadhyay, A. Banerjee, D. K. Harnal, W. N. Kagitha, R. Prasad, S. Singh, A. Sharma, P. Duttgupta, and M. Lohia, “Mem0: Building production-ready AI memories for LLM applications,” arXiv:2504.19413, 2025.
- [6] Z. Chen, X. Liu, X. Li, F. Zhou, C. Fu, and L. Li, “MemoryOS: A multimodal memory system for AI applications,” arXiv:2506.06326, 2025.
- [7] T. Vlašić and T. Horvat, “MIRIX: A multi-agent system architecture for long-term memory,” arXiv:2507.07957, 2025.
- [8] Z. Zhu, R. R. Martinez, C. Zhou, Z. Li, and X. S. Lin, “Graphiti: Incremental temporal knowledge graph construction for evolving world state,” arXiv:2501.13956, 2025.
- [9] Z. Luo, J. He, S. Dai, R. Ma, B. Wang, and Y. Deng, “LightMem: A lightweight memory system for long-context LLM agents,” in *Proc. ICLR*, 2026.
- [10] J. Liu, Y. Su, P. Xia, S. Han, Z. Zheng, C. Xie, M. Ding, and H. Yao, “SimpleMem: Efficient lifelong memory for LLM agents,” arXiv:2601.02553, 2026.
- [11] J. van den Brandt, F. Stolp, T. de Groot, and J. F. Santos, “Hindsight: Post-hoc memory-based learning for LLM agents,” in *Proc. ACL System Demonstrations*, 2026.
- [12] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, “MemSkill: Learning and evolving memory skills for self-evolving agents,” arXiv:2602.02474, 2026.
- [13] J. Liu, X. Ye, P. Xia, Z. Zheng, C. Xie, M. Ding, and H. Yao, “EvolveMem: Self-evolving memory architecture via AutoResearch for LLM agents,” arXiv:2605.13941, 2026.
- [14] MemMA contributors, “MemMA: A meta-thinking approach for joint coordination of memory construction and access in LLM agents,” arXiv:2603.18718, 2026.
- [15] PlugMem contributors, “PlugMem: A task-agnostic plug-in memory module for LLM agents,” arXiv:2603.03296, 2026.
- [16] S. Ji and colleagues, “MRAgent: Active memory reconstruction with a graph-based retrieval agent for long-term LLM agents,” in *Proc. ICML*, 2026.

- [17] MemMachine contributors, “MemMachine: Ground-truth-preserving episodic memory for LLM agents,” arXiv:2604.04853, 2026.
- [18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023.
- [19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dy, S. Gupta, A. Borgida, K. Sridhar, W. Yih, and D. Roth, “Self-Refine: Iterative refinement with self-feedback,” in *Proc. NeurIPS*, 2023.
- [20] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” in *Proc. AAAI*, 2024.
- [21] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” in *Proc. ICLR*, 2024.
- [22] LangChain AI, “LangMem SDK: Long-term memory for LangGraph agents,” <https://github.com/langchain-ai/langmem>, 2025.
- [23] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *TMLR*, 2024.
- [24] A. Maharana, D. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of LLM agents,” in *Proc. ACL*, 2024.
- [25] D. Wu, C. Zhao, Y. Ma, and S. Krishnan, “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” arXiv:2410.10813, 2024.